

Módulos y Paquetes en Python

Organización, reutilización y distribución de código

`import`

`from ... import`

`__init__.py`

`pip`

`namespace`

Contenido de la presentación

01

¿Qué es un módulo?

Concepto y creación de módulos en Python

02

Importar módulos

Diferentes formas de usar import

03

Módulos de la librería estándar

os, sys, math, datetime y más

04

¿Qué es un paquete?

Estructura de directorios y `__init__.py`

05

Paquetes de terceros y pip

Instalar y gestionar dependencias

06

Ejercicios prácticos

Manos a la obra con código real

¿Qué es un módulo?

CONCEPTO

Un módulo es un archivo .py que contiene definiciones de funciones, clases y variables que pueden ser reutilizadas en otros programas.



Archivo .py

Cualquier archivo Python es un módulo



Reutilizable

Importa código en múltiples proyectos



Encapsulado

Agrupar código relacionado en un lugar

EJEMPLO

```
# archivo: matematicas.py

def sumar(a, b):
    """Retorna la suma de dos números."""
    return a + b

def multiplicar(a, b):
    """Retorna el producto de dos números."""
    return a * b
```

Importar módulos – formas de uso

`import módulo`

Importa todo el módulo. Acceso con prefijo.

```
import matematicas

resultado = matematicas.sumar(3, 5)
print(resultado)    # 8
```

`from módulo import nombre`

Importa elementos específicos. Sin prefijo.

```
from matematicas import sumar, PI

resultado = sumar(3, 5)
print(PI)    # 3.14159265
```

`import módulo as alias`

Importa con alias para nombres más cortos.

```
import matematicas as mat

resultado = mat.multiplicar(4, 7)
print(resultado)    # 28
```

`from módulo import *`

Importa todo (⚠ evitar en prod).

```
from matematicas import *

resultado = multiplicar(4, 7)
# Puede causar conflictos de nombres
```

Librería Estándar de Python

Python incluye más de 200 módulos listos para usar. ¡No necesitas instalar nada!



math

Funciones matemáticas

```
math.sqrt(16) → 4.0
```



os

Sistema operativo

```
os.getcwd() → ruta
```



sys

Intérprete Python

```
sys.argv, sys.exit()
```



datetime

Fechas y tiempo

```
datetime.now()
```



random

Números aleatorios

```
random.randint(1,10)
```



json

Lectura/escritura JSON

```
json.dumps({}) json.loads()
```

¿Qué es un paquete?

Un paquete es un directorio que contiene múltiples módulos y un archivo especial `__init__.py` que indica a Python que esa carpeta es un paquete.

ESTRUCTURA

```
mi_paquete/  
├── __init__.py      ← hace que sea un paquete  
├── matematicas.py  
├── estadistica.py  
└── utils/  
    ├── __init__.py  
    ├── formato.py  
    └── validacion.py
```

`__init__.py`

¿Para qué sirve `__init__.py`?

- Marca el directorio como paquete
- Puede importar submódulos
- Puede definir variables públicas (`__all__`)
- Puede ejecutar código de inicialización

IMPORTAR DESDE UN PAQUETE

```
# Importar un módulo del paquete  
import mi_paquete.matematicas as mat  
  
# Importar función específica  
from mi_paquete.estadistica import promedio
```

Paquetes de terceros · pip

¿QUÉ ES pip?

pip es el gestor de paquetes de Python. Permite instalar, actualizar y eliminar paquetes desde PyPI (Python Package Index).

COMANDOS ESENCIALES

`pip install nombre`

Instala un paquete

`pip install nombre==1.2.3`

Instala versión específica

`pip install -r requirements.txt`

Instala desde archivo

`pip uninstall nombre`

Desinstala un paquete

`pip list`

Lista paquetes instalados

`pip show nombre`

Información de un paquete

requirements.txt

```
# requirements.txt
numpy==1.26.0
pandas>=2.0.0
requests~=2.31.0
matplotlib
```

PAQUETES POPULARES

<code>numpy</code>	Cálculo numérico
<code>pandas</code>	Análisis de datos
<code>requests</code>	HTTP
<code>flask</code>	Web apps
<code>matplotlib</code>	Gráficas

Ejercicio 1 · Crear tu primer módulo



EJ. 1

Objetivo: crear un módulo de operaciones con cadenas y usarlo en otro archivo.

PASO 1 · crear cadenas.py

```
# cadenas.py

def invertir(texto):
    """Retorna el texto al revés."""
    return texto[::-1]

def contar_vocales(texto):
    """Cuenta las vocales en el texto."""
    vocales = "aeiouáéíóúAEIOUÁÉÍÓÚ"
    return sum(1 for c in texto if c in vocales)

def capitalizar_palabras(texto):
    """Capitaliza cada palabra."""
    return texto.title()
```

PASO 2 · usar el módulo

```
# main.py

import cadenas

# Usar las funciones
texto = "hola mundo python"

print(cadenas.invertir(texto))

print(cadenas.contar_vocales(texto))

print(cadenas.capitalizar_palabras(texto))
```


Ejercicio 2 · Construir un paquete



EJ. 2

Objetivo: crear un paquete calculadora con tres módulos: basica, avanzada y utilidades.

ESTRUCTURA

```
calculadora/  
├── __init__.py  
├── basica.py  
├── avanzada.py  
└── utilidades.py
```

basica.py

```
def sumar(a, b): return a + b  
def restar(a, b): return a - b  
def multiplicar(a, b): return a * b  
def dividir(a, b):  
    if b == 0:  
        raise ValueError("No dividir por  
0")  
    return a / b
```

avanzada.py

```
import math  
def potencia(base, exp): return base **  
exp  
def raiz(n): return math.sqrt(n)  
def factorial(n): return  
math.factorial(n)  
def log(n, base=10): return math.log(n,  
base)
```

__init__.py

```
# calculadora/__init__.py  
from .basica import sumar, restar, multiplicar, dividir  
from .avanzada import potencia, raiz, factorial  
  
__version__ = "1.0.0"  
__all__ =  
["sumar", "restar", "multiplicar", "dividir", "potencia", "raiz", "  
factorial"]
```

USO DEL PAQUETE

```
import calculadora  
  
print(calculadora.sumar(5, 3))  
print(calculadora.potencia(2, 10))  
print(calculadora.factorial(6))  
print(calculadora.__version__)  
  
from calculadora.avanzada import log  
print(log(1000))
```

Ejercicio 3 · Librería estándar y paquetes externos



EJ. 3

PARTE A · Módulos estándar

```
import os, sys, datetime, random, json

# 1. Información del sistema
print(f"Python: {sys.version}")
print(f"Directorio: {os.getcwd()}")
print(f"Archivos: {os.listdir('.')}")

# 2. Fecha y hora actuales
ahora = datetime.datetime.now()
print(f"Hoy: {ahora.strftime('%d/%m/%Y %H:%M')}")

# 3. Números aleatorios y estadísticas
muestra = [random.randint(1, 100) for _ in range(10)]
print(f"Muestra: {muestra}")

# 4. Trabajar con JSON
datos = {"nombre": "Python", "version": 3, "activo": True}
json_str = json.dumps(datos, indent=2)
print(json_str)
recuperado = json.loads(json_str)
print(type(recuperado)) # <class 'dict'>
```

PARTE B · Paquete requests

```
# Instalar primero:
# pip install requests

import requests

# Obtener datos de una API pública
url =
"https://jsonplaceholder.typicode.com/todos/1"
respuesta = requests.get(url)

if respuesta.status_code == 200:
    dato = respuesta.json()
    print(f"ID: {dato['id']}")
    print(f"Título: {dato['title']}")
    print(f"Completado: {dato['completed']}")
else:
    print(f"Error: {respuesta.status_code}")
```

Resumen y Buenas Prácticas



Módulo = archivo .py

Cualquier archivo Python con código reutilizable



Paquete = carpeta + __init__.py

Agrupar módulos relacionados en un directorio



import / from ... import

Diferentes formas de acceder al código



Librería estándar

Más de 200 módulos listos, no necesitas instalar



pip install

Instala paquetes externos desde PyPI



Buenas Prácticas



Un módulo, una responsabilidad



Nombres en minúsculas con guiones bajos:
mi_modulo.py



Documenta funciones con docstrings



Evita import * en producción



Usa requirements.txt para dependencias



Organiza imports: estándar → terceros → propios



Prueba tus módulos antes de distribuirlos

¡Ahora organiza tu código!

Módulos + Paquetes = código limpio, reutilizable y escalable

Módulos .py

import / from

Librería estándar

Paquetes

pip install

docs.python.org · pypi.org · realpython.com

MÓDULOS Y PAQUETES EN PYTHON

Organiza, reutiliza y potencia tu código 🚀



Python tiene muchas funcionalidades integradas y también permite que nosotros creamos las nuestras. Para mantener todo ordenado, usa módulos y paquetes.

¿QUÉ ES UN MÓDULO?

Un módulo es un archivo .py que contiene código Python: funciones, variables, clases, etc., que puedes usar en otros programas.



Ejemplo:

- math.py
- funciones_utiles.py
- mi_modulo.py

¿CÓMO SE USA?

Usas la palabra clave `import` para traer el módulo a tu programa.

```
# Importo todo el módulo math
import math
print(math.sqrt(16)) # 4.0

# Importo solo una función
from math import pi
print(pi) # 3.141592653589793
```

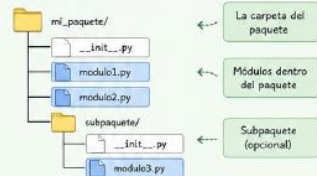


Python ya incluye muchos módulos útiles, como `math`, `random`, `datetime`, `os`, `sys`, etc.

¿QUÉ ES UN PAQUETE?

Un paquete es una carpeta que contiene módulos y (opcionalmente) subpaquetes.

Permite organizar proyectos grandes en estructuras jerárquicas.



¿CÓMO SE USA?

Importas módulos dentro del paquete usando la notación de punto.

```
# Importar un módulo del paquete
from mi_paquete import modulo1
modulo1.mi_funcion()

# Importar desde un subpaquete
from mi_paquete.subpaquete import modulo3
modulo3.otra_funcion()
```

EJEMPLOS VISUALES

MÓDULO (archivo único)



```
def saludar(nombre):
    return f"¡Hola, {nombre}!"
```

Uso en otro programa:

```
import saludo
print(saludo.saludar("Ana")) # ¡Hola, Ana!
```

PAQUETE (carpeta con)



Uso en otro programa:

```
from herramientas import texto, fechas
print(texto.contar_palabras("Hola mundo")) # 2
print(fechas.hoy()) # 2024-05-20
```

```
# herramientas/texto.py
def contar_palabras(s):
    return len(s.split())

# herramientas/fechas.py
from datetime import date
def hoy():
    return date.today()
```

DIFERENCIAS CLAVE

MÓDULO

- Es un archivo .py
- Contiene código relacionado en un solo archivo
- Se importa con `import` o `from ... import ...`
- Útil para funcionalidades específicas y reutilizables

PAQUETE

- Es una carpeta que contiene módulos y subpaquetes
- Organiza código en una estructura jerárquica
- Se importa usando la notación de punto
- Ideal para proyectos grandes y escalables

¡RECUERDA!

- ✓ Cada paquete debe tener un archivo `__init__.py` (puede estar vacío).
- ✓ Usa nombres claros para tus módulos y paquetes.
- ✓ Organizar tu código hace tus programas más fáciles de entender y mantener.

BUENAS PRÁCTICAS

Usa módulos para reutilizar código en diferentes programas.

Usa paquetes para mantener tu proyecto ordenado.

Evita nombres que choquen con módulos existentes (ej: `math.py`).

Comenta tu código y documenta tus módulos (y funciones).